

ISEC6000 Secure DevOps

Assessment 1 DevOps CI/CD Pipeline

Weight: Refer to the Unit Outline

Deadline: Refer to the Unit Outline / LMS Submission Point

Late Penalty:

1. For assessment items submitted within the first 24 hours after the due date/time, students will be penalised by a deduction of 5% of the total marks allocated for the assessment task;
2. For each additional 24 hour period commenced an additional penalty of 10% of the total marks allocated for the assessment item will be deducted; and
3. Assessment items submitted more than 168 hours late (7 calendar days) will receive a mark of zero.

Generative AI policy:

As outlined in the Unit Outline, “You are not permitted to use Gen-AI software [excluding approved tools such as Curtin-provided Grammarly] in any assessment task”. For any suspected use of unauthorised Gen-AI software, students may be required to attend a viva examination to demonstrate and verify their understanding of the submitted work.



Updated: 28 July 2026

1. Project Overview

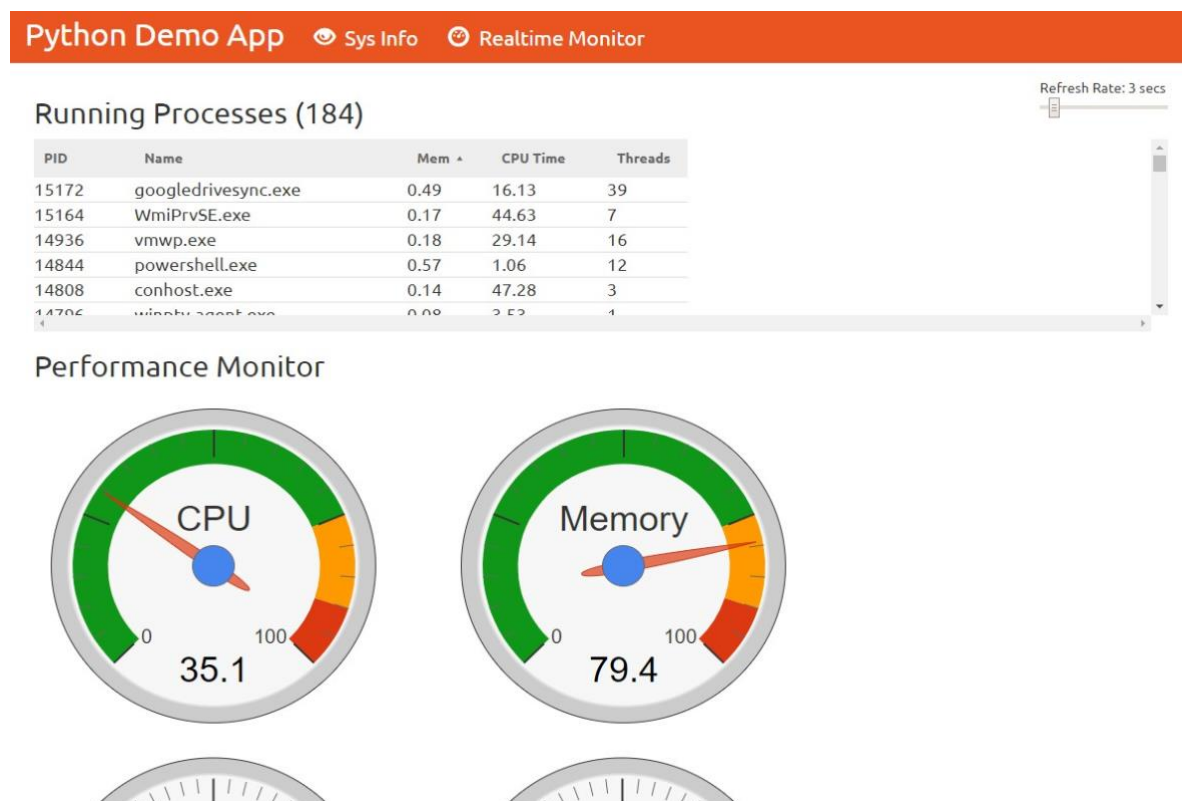
In today's software industry, automation is the backbone of security and reliability. Every major tech company, e.g., from Netflix to Atlassian to Canva, relies on Secure DevOps pipelines to deliver code quickly, safely, and at scale. Instead of "deploying once a month", modern teams deploy hundreds of times per day, with every change tested, scanned, containerised and shipped automatically.

This assignment gives you a hands-on journey into that world. You will start small, running local tests, and then gradually grow your skills to:

- **Automate builds** so your application compiles and tests itself
- **Containerise your app** with Docker to ensure "it runs everywhere"
- **Secure your pipeline** with GitLab's variables and secrets management
- **Publish your work** to Docker Hub and deploy it to a real environment

By the end, you'll have your own **working CI/CD pipeline**, just like a professional DevOps engineer. It is the same workflow you would set up in a startup, an enterprise software team, or even in open-source projects.

Think of this project as your **first personal DevOps portfolio item**: something you can proudly demo in interviews or on GitHub/GitLab to show employers that you understand how modern software delivery really works.



2. Learning Goals

This project introduces you to **end-to-end CI/CD with GitLab**, applied to a simple Python web application. You will:

- Understand GitLab pipelines and how they differ from Jenkins and other CI/CD tools.
- Learn the key building blocks: **jobs, stages, runners, variables**.
- Automate:
 1. Running unit tests
 2. Building & tagging Docker images
 3. Pushing images to a private Docker Hub repository
 4. Managing credentials with GitLab CI/CD variables
- Document your steps with **clear explanations and screenshots** to demonstrate competency.

3. Task 1: Run and Test a Python Demo App (Local)

You will use the demo repository python-demoapp (<https://github.com/benc-uk/python-demoapp>), a small Python web app that displays system information. In this stage, you do not need to understand the internal code, only focus on **running, testing, and deploying** it.

Steps

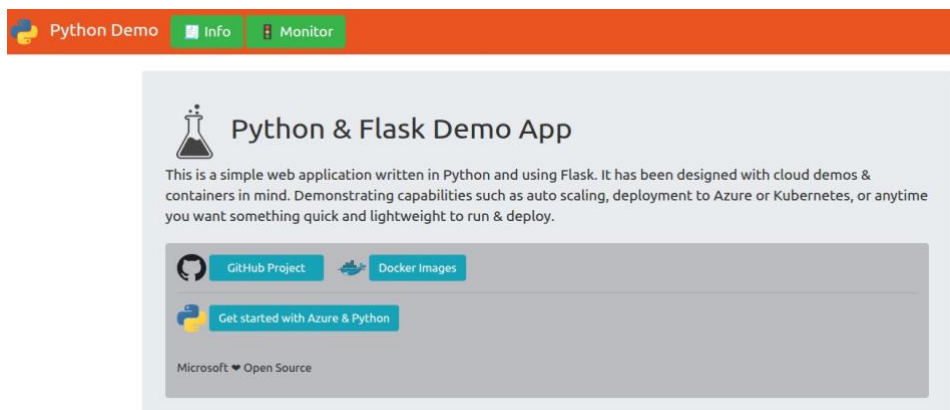
- 3.1. Clone the following updated/modified repository locally:
<https://github.com/nurcurtin/ISEC60001>
- 3.2. Explore the project structure: especially, look at src/ and tests/ folders.
- 3.3. Run tests locally using Makefile (test)

If any errors appear, resolve them through standard practice, such as upgrading packages, cleaning caches, etc.

- 3.4. Start the app using command from Makefile (run)

Ensure it runs on **port 5004** (not the default 5000).

- 3.5. Open the link in a browser, which would look something like this:



- 3.6. Replace the access URL with your local IP and report it in the format:
`http://Your_Server_IP:5004/`

Share the URL in the report and give some evidence (screenshot). We may access the URL to check whether the application is working or not.

4. Task 2: Incremental GitLab CI/CD Pipeline

You will now create an incremental pipeline on GitLab. Each step builds upon the previous one.

Steps

Step 1: Create a GitLab Project

- Create a new project in your GitLab.
- Push the current codebase (including changes introduced in Task 1) to this repository.

The `ci/` directory contains multiple YAML files, each outlining specific instructions for build and test operations, which you may need in various stages of this assessment.

Step 2. Basic Pipeline

- Use `step2-ci.yml`, and modify the pipeline to print:
Hi all, my name is <Your Name>, my student id is <Your ID>.
- Run pipeline and include related logs screenshot in your report.

Step 3. Docker Build & Push (hardcoded credentials)

- Requirements: A Docker Hub account and a repository at <https://hub.docker.com/>
- Use `step3-ci.yml`:
 - o Add your Docker Hub *username/password*
 - o Add proper *ImageName:ImageTag* with your repo
- Run pipeline → screenshot of logs + Docker Hub repo with pushed image.
- Mask credentials in report screenshots. (Do not share your dockerhub credential in the report.)

Step 4. Secure with GitLab Variables

- Use `step4-ci.yml`.
- Create two CI/CD variables in GitLab: `REGISTRY_USER`, `REGISTRY_PASS`. The value of these two variables should contain the credentials of your docker hub.
- Also use `$REGISTRY_USER / $REGISTRY_PASS` in your GitLab CI.
- Run pipeline → screenshot logs (showing successful login & push).

Step 5. Use Non-Sensitive Variables

- Use step5-ci.yml.
- Add reusable variables (e.g., IMAGE_NAME, IMAGE_TAG) in your GitLab CI.
- Run pipeline → screenshot logs and image tags on Docker Hub.

Step 6. Finalised Pipeline

- Use step6-ci.yml.
- Follow inline comments to finalise the pipeline.
- Provide logs & screenshots of final successful pipeline.

5. Report & Video Project Demo Submission Guidelines

- **Cover Page:**
 - Full Name
 - Student ID
 - GitLab repo link (public)
 - Any other relevant URLs (e.g., Docker Hub repo)
 - Link to your recorded screen demo showcasing the project in action (Max duration: 10min).
 - You must include a webcam view ("talking head") of yourself in a corner of the video throughout the demonstration.
 - The video should demonstrate:
 - The running application
 - The GitLab repository
 - Successful pipeline execution
 - Docker Hub image publication
 - A brief explanation of the final CI/CD workflow
 - You can upload the video anywhere you like such as OneDrive, Google Drive, YouTube, and include the sharable link here.
 - **It is your responsibility to ensure the link is accessible to teaching staff (e.g., Anyone with the link can view).**
- **Structure:** *(including but not limited to the following)*
 - Task 1: Subtasks with explanations & screenshots
 - Screenshot of successful test execution in the terminal, including any relevant logs or configuration settings.
 - Screenshot of the running application in a web browser, clearly showing the local IP address and server URL.
 - A concise description of any troubleshooting steps taken during the task.
 - Task 2: Step-by-step pipelines (subtasks 1 – 6), with evidence of completion
 - Evidence may include pipeline logs, GitLab screenshots, Docker Hub screenshots, and brief explanations as appropriate.

- **Style Tips:**
 - Explain **why** each step is done.
 - Clearly highlight **before/after** pipeline changes.
 - Explanations must be written in your own words and clearly demonstrate your understanding of the CI/CD process.
 - Use **consistent formatting** for commands (code blocks).
 - Add bookmarks/headings in your PDF for easy navigation.
 - Only placing screenshots without associated discussion is not sufficient. Each screenshot must include:
 - A figure number and caption
 - A brief explanation of what the screenshot demonstrates
 - Accompanying discussion in the report text.
- **Submission Filename:**
 - Filename: *P1_StudentID.pdf*.